Digital Testing and verification for SPI IP slave module

Petro Nazeh Ozores Ghali V23010552

Youssef Mohamed Abdul-Satar. V23010256

Hussein Muhammad Ahmad Albaqar. V23010547

Rofida Aymen Abdelhamid V23010126

Mahmoud Rabie Al-Shall. V23010514

Supervisor : TA/ENG Hassan

Section 10 - CND211_Group 5

15/5/2024

Contents

1. **Introduction**.....3
   1.1 SPI protocol overview.....3
   1.2 Analysis SPI slave IP.....5
   1.3 Tools used in simulation.....6
   1.4 Verification plan.....7
2. **Design**.....8
   2.1 SPI slave RTL design module.....9
   2.2 SPI slave Interface.....9
   2.3 SPI slave Class 10
   2.4 SPI slave Code Coverage.....11
   2.5 SPI slave Program test.....12
   2.6 SPI slave Top Testbench.....14
3. **Testing & Simulation**.....15
4. **Refrences**.....18

1. Introduction

1.1 SPI protocol overview

Serial Peripheral Interface (SPI) is a synchronous serial communication protocol used for short-distance communication, primarily in embedded systems. Here is an overview of the SPI protocol:

- **Master-Slave Architecture**: SPI operates in a master-slave configuration. The master device initiates the communication and controls the clock, while one or more slave devices respond.

- **Four Main Signals**:
  - **MOSI (Master Out Slave In)**: Carries data from the master to the slave.
  - **MISO (Master In Slave Out)**: Carries data from the slave to the master.
  - **SCLK (Serial Clock)**: Clock signal generated by the master to synchronize data transfer.
  - **SS (Slave Select)**: An active-low signal used by the master to select which slave device to communicate with.

- **Full-Duplex Communication**: Data can be transmitted and received simultaneously, with data shifted out on the MOSI line and shifted in on the MISO line during the same clock cycle.

- **Clock Polarity and Phase**: SPI allows for different clock configurations, which are defined by two parameters:
  - **Clock Polarity (CPOL)**: Determines the idle state of the clock signal (low or high).
  - **Clock Phase (CPHA)**: Determines when data is sampled (on the leading or trailing edge of the clock).

- These configurations result in four possible modes (Mode 0, Mode 1, Mode 2, Mode 3).

- **Simplex, Half-Duplex, and Full-Duplex**: SPI supports all three modes of communication. However, full-duplex is the most common.

- **Speed**: SPI can operate at high speeds, typically up to several MHz, depending on the devices and the system requirements.

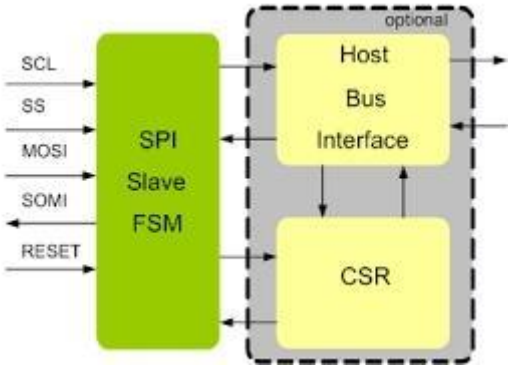
- **No Acknowledgment**: Unlike protocols like I2C, SPI does not have an acknowledgment mechanism, which means the master cannot confirm if the slave has correctly received the data.

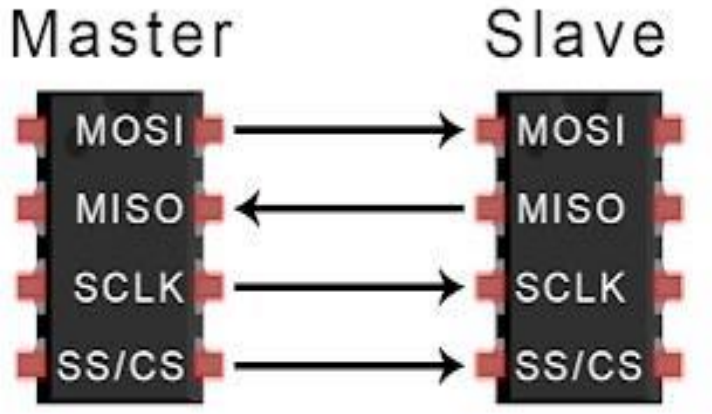
- **Multiple Slaves**: Multiple slave devices can be connected to a single master by using multiple SS lines or a daisy-chain configuration.

- **Applications**: SPI is commonly used in applications requiring high-speed data transfer, such as communication with sensors, SD cards, and other peripherals in embedded systems.

- SPI is valued for its simplicity, speed, and versatility, making it a popular choice for many embedded applications.

1.2 Analysis of SPI slave IP





The SPI slave IP implementation used in this design consists of 4 main pins which and 5 for transaction functionality

Implementation pins	Type	Functionality
MOSI	**1-bit input data port**	**Receive data from master serialized to slave**
MISO	**1-bit output data port**	**Send data from slave to master serialized**
SS(CS)	**1-bit input signal**	**Signal to select which slave IP device to read/write (active low)**
SCLK	**1-bit input signal**	**CIK generated from master to slave**
reset	**1-bit input signal**	**Signal to reset the slave IP device (active low)**
SlaveDatatoSend	**8-bit input data port**	**Data to be send on miso**
SlaveDataReceived	**8-bit output data port**	**Received 8-bit data from MOSI to parallel mode**
wr_en	**1-bit input signal**	**Signal for enable write in slave ip**
rd_en	**1-bit input signal**	**Signal for read from slave data to master**

1.3 Verification plan

A verification plan in digital design is a detailed document outlining the strategy and steps to ensure that a digital design meets its specifications and functions correctly. This plan typically includes

Test Plan: Lists the test cases to be executed, covering all functional scenarios, corner cases, and performance requirements.

Design Under Test (DUT): Specifies the module or system being verified

Methodology: Outlines the verification methodologies to be used, such as directed testing, random testing, coverage-driven verification, and formal verification.

Tools and Resources: Identifies the simulation tools, hardware platforms, and other resources required for verification.

Pass/Fail Criteria: Specifies the criteria for determining whether the design has passed or failed the verification process.

In this design the following verification plan applied

Testing and verify : SPI slave IP module

Methodologies used : randomization testing , assertions and coverage of signals/ports

Constrains :

- CLK generated with this configuration 1ns rise and 2ns fall time
- Reset signal is asserted to 1 for 1ns and 0 for rest of read/write cycle (active low)
- MOSI and MISO are hi-z in case of CS is high or no rd/wr
- Read/Write cycles each cycle should be finished after 10 CLK cycles

  First CLK for starting transaction reset will be high then go low

  8 CLKs for sending/receiving data to/from Slave device

  Last CLK used for finishing the transaction data should be read/wrote
- SS (CS or chip select) signal should be high when reset is asserted to high for 6ns and then 0 for rest of read/write cycle (active low)

Implementation and Transaction operating modes : the SDS (slave data to send) and MDS(master data to send) are randomized as well ass wr_en and rd_en with No constrains because spi can work in full doplex mode do both read/write operation in same time **modes 1. Read 2. Write 3. Full doplex**

Assertions: in the last CLK data to be checked in testbench if transferred correctly from source to destination MDS,Recieve are assumed to be in/out parallel ports Master and SlaveDatatoSend, SlaveDataReceived for slave IP transaction starting Data start moving from slavedatatosend through MISO bus to Receive port of master in reading operation and data in MDS should go through MOSI to SlaveDataReceived

Coverage: all signals and ports should cover all possible test cases in design

1.4 Tools used in simulation

- **VCS** is a Verilog simulator and compiler tool from Synopsys used for RTL (Register Transfer Level) simulation and synthesis.
- **Verdi** is a debugging tool from Synopsys used for debugging and analyzing RTL and gate-level simulation results.
- **DVE** (Design and Verification Environment) is a debugging tool from Cadence used for debugging RTL designs
- **Eda playground (vcs)** eda playground provides also good environment for testing and simulation on vcs
- **Linux environment** tools are used in Linux and running simulation is done through terminal

2. Design

2.1 SPI Slave RTL design module

```
module Slave
(
        input reset,
        input [7:0]slaveDataToSend,
        output [7:0]slaveDataReceived,
        input SCLK, CS, MOSI,
    input wr_en ,rd_en,
        output reg MISO
);
integer counter;
reg [7:0] SDS;
reg [7:0] SDR;

wire [7:0] SDS_next;
wire [7:0] SDR_next;

always @ (negedge CS)
begin
        counter = 0;
        SDS = slaveDataToSend;
end
always @(posedge SCLK , posedge reset)
begin
if(reset==1'b1)
begin
        SDS=slaveDataToSend;
        SDR=0;
        counter = 0;
end
else
begin
        SDR=SDR_next;
        SDS=SDS_next;
        counter = counter+1;
end
end
 assign SDS_next = (CS==1'b0&&counter<=8 &&counter >0 && rd_en)?{SDS[6:0],1'b0 }:SDS;
 assign SDR_next = (CS==1'b0&&counter<=8 && wr_en )?{SDR[6:0],MOSI}:SDR;
assign MISO = (CS==1'b0 && rd_en)? SDS[7]:1'bz;
assign slaveDataReceived = SDR;
endmodule
```

2.2 SPI slave Interface

```
// Define interface
interface intf;
    logic reset;
    logic [7:0] slaveDataToSend;
    logic [7:0] slaveDataReceived;
    logic SCLK, CS, MOSI;
    logic wr_en ,rd_en;
    logic MISO;

    clocking spi_clocking @(posedge SCLK);
        input reset;
        output MOSI, CS;
        input MISO;
        input wr_en ,rd_en;
    endclocking
    endinterface
```

2.3 SPI slave class

```
`define MAX_ITERATIONS 10



// Define SlaveIP class
class SlaveIP;
    // Define variables
    rand bit [7:0] DataSend[0:`MAX_ITERATIONS - 1];
    rand bit [7:0] SDS[0:`MAX_ITERATIONS - 1];
    bit [7:0] MDS;
    bit [7:0] Recieve;
    virtual intf spi_intf;

    // Constructor to initialize variables
    function new();
        // Randomize DataSend and SDS arrays
        foreach (DataSend[i])
          DataSend[i] = $urandom;
        foreach (SDS[i])
          SDS[i] = $urandom;
    endfunction

   constraint mosi_miso_constraint {
     if(spi_intf.CS == 1)
       spi_intf.MOSI == spi_intf.MISO;
   }
          endclass
```

2.4 SPI slave code coverage

```
module slaveCoverage(intf spi_intf);
 // Coverage group definition
   covergroup spi_coverage;
   option.per_instance = 1; // Allows per-instance coverage

   // Coverage points for Full and Empty flags
     coverpoint(spi_intf.MOSI) {
       bins MOSI_is_true = {1'b1}; // Bin for full being true
       bins MOSI_is_false = {1'b0}; // Bin for full being false
   }

     coverpoint(spi_intf.MISO) {
       bins MISO_is_true = {1'b1}; // Bin for empty being true
       bins MISO_is_false = {1'b0}; // Bin for empty being false
   }

     coverpoint(spi_intf.wr_en) {
       bins wr_is_true = {1'b1}; // Bin for empty being true
       bins wr_is_false = {1'b0}; // Bin for empty being false
   }

     coverpoint(spi_intf.rd_en) {
       bins rd_is_true = {1'b1}; // Bin for empty being true
       bins rd_is_false = {1'b0}; // Bin for empty being false
   }
   endgroup

    spi_coverage fc = new;
        endmodule
```

2.5 SPI slave Program Test

```
program test_program(input SCLK);

// Instantiate the slave IP class
    SlaveIP slave_inst = new();


// Define variables
integer i, j; // Declare i and j here
bit [7:0] MDS;
bit [7:0] Recieve;
int numITerations = `MAX_ITERATIONS;

task write_task();
    int i;
    spi_intf.MOSI = slave_inst.MDS[7];
    if(!spi_intf.wr_en)
        spi_intf.MOSI = 1'bz;
    #2
    for (i = 6; i >= 0; i = i - 1) begin
        #1;
        spi_intf.MOSI = slave_inst.MDS[i];
        if(!spi_intf.wr_en)
        spi_intf.MOSI = 1'bz;
        #2;
    end
    #1;
endtask

task read_task();
    int i;
    #2
  for (i = 6; i >= 0; i = i - 1) begin
        #1;
        slave_inst.Recieve[i+1] = spi_intf.MISO;
        #2;
    end
    #1;
    slave_inst.Recieve[0] = spi_intf.MISO;
    Recieve =  slave_inst.Recieve;
endtask
```

```
// Testbench code
initial begin
    spi_intf.SCLK = 0;
    spi_intf.MOSI = 1'bz;
    spi_intf.reset = 1;
    // Test data initialization
    for (j = 0; j < numITerations; j = j + 1) begin
        slave_inst.MDS = slave_inst.DataSend[j];
        MDS = slave_inst.MDS;
        spi_intf.slaveDataToSend = slave_inst.SDS[j];
        spi_intf.CS = 1;
        #6
        spi_intf.CS = 0;

        fork
        write_task();
        read_task();
        join
        disable fork;

          // Assert slaveDataToSend and slaveDataReceived signals
          if(spi_intf.rd_en) begin
        assert (slave_inst.Recieve == spi_intf.slaveDataToSend)
          $display("Master received Data %x from slave successfully",slave_inst.Recieve);
            else $error("slaveDataSent FAILED");
          end

          if(spi_intf.wr_en) begin
        assert (spi_intf.slaveDataReceived == slave_inst.MDS)
          $display("Slave received Data %x from Master successfully",spi_intf.slaveDataReceived );
            else $error("slaveDataReceived FAILED");
          end
    end
    $finish;
end

endprogram
```

2.6 SPI slave Top testbench

```
module Slave_tb();

// Instantiate the interface
intf spi_intf();
// Instantiate the slave module
Slave M1(spi_intf.reset, spi_intf.slaveDataToSend, spi_intf.slaveDataReceived,
        spi_intf.SCLK, spi_intf.CS, spi_intf.MOSI, spi_intf.wr_en, spi_intf.rd_en, spi_intf.MISO);

  //CLOCK Generation
  always begin
#2
  spi_intf.SCLK = 1;
#1
  spi_intf.SCLK = 0;
end
//MOSI should be hi-z if no slave selection provided
  always @ (posedge spi_intf.CS)
begin
  spi_intf.MOSI = 1'bz;
  spi_intf.rd_en = $urandom_range(1,0);
  spi_intf.wr_en = $urandom_range(1,0);
end
//Generate reset signal
always begin
#1
spi_intf.reset = 0;
if(M1.counter == 4'b1000)
#1
spi_intf.reset = 1;
end

 test_program tb(
   .SCLK(spi_intf.SCLK) // Connect the clock signal
  );

  slaveCoverage sv(spi_intf);

// Dump waveform
initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(0, Slave_tb);
end
    endmodule
```

3. Testing & Simulation

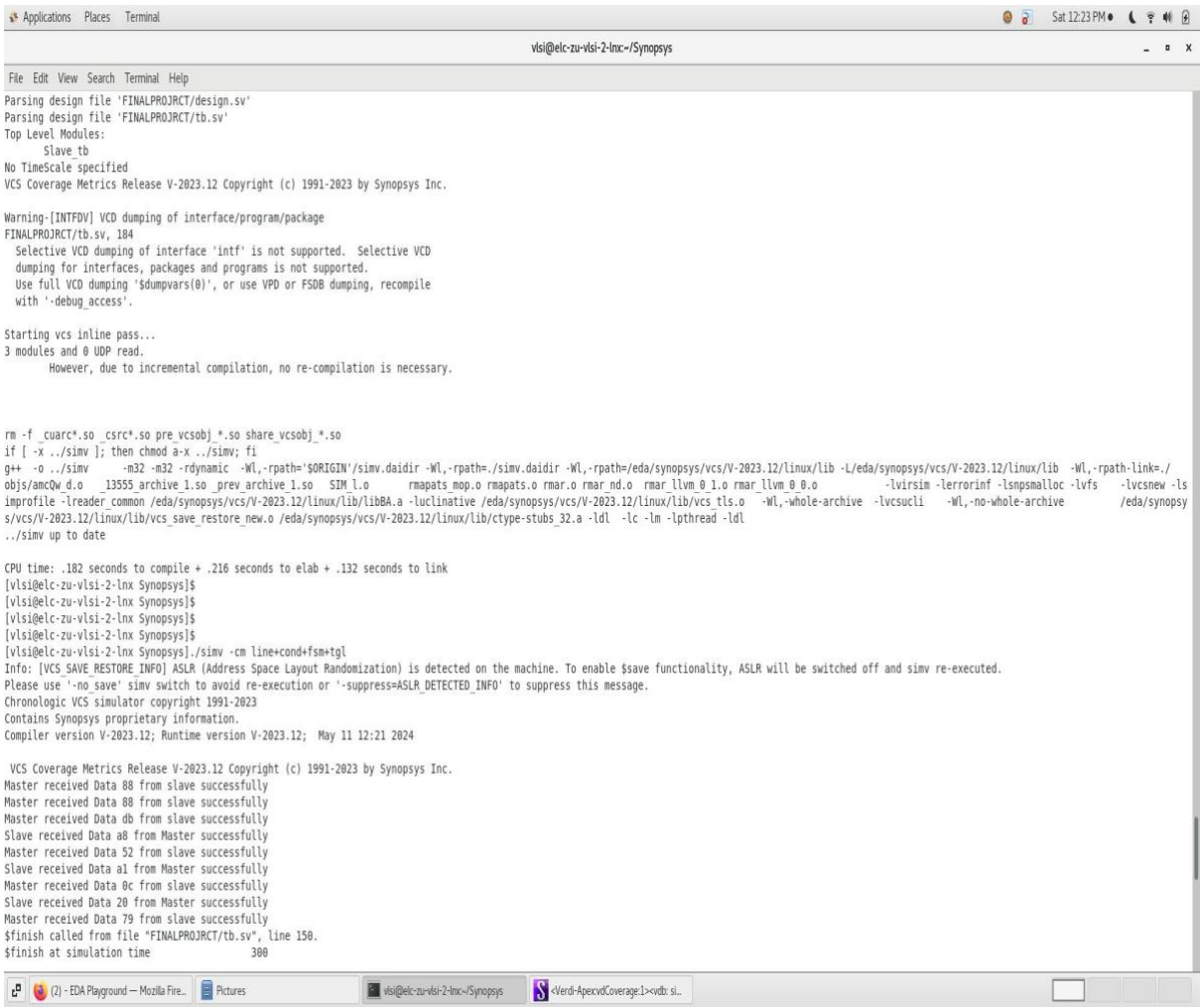
Running VCS simulation is done in a linux environment
Tested in CentOS 7 distro copy all files to synopsis folder
Open terminal in synopsis path

1. Compiling the code with vcs

   vcs -sverilog -ntb_opts dtm -cm line+cond+fsm+tgl Slaveip_Top.sv Slaveip-RTLdesign.v Slaveip_Class.sv Slaveip_Interface.sv Slaveip_Test.sv Slaveip_Coverage.sv

2. Running simulation with coverage reporting support

   ./simv -cm line+cond+fsm+tgl

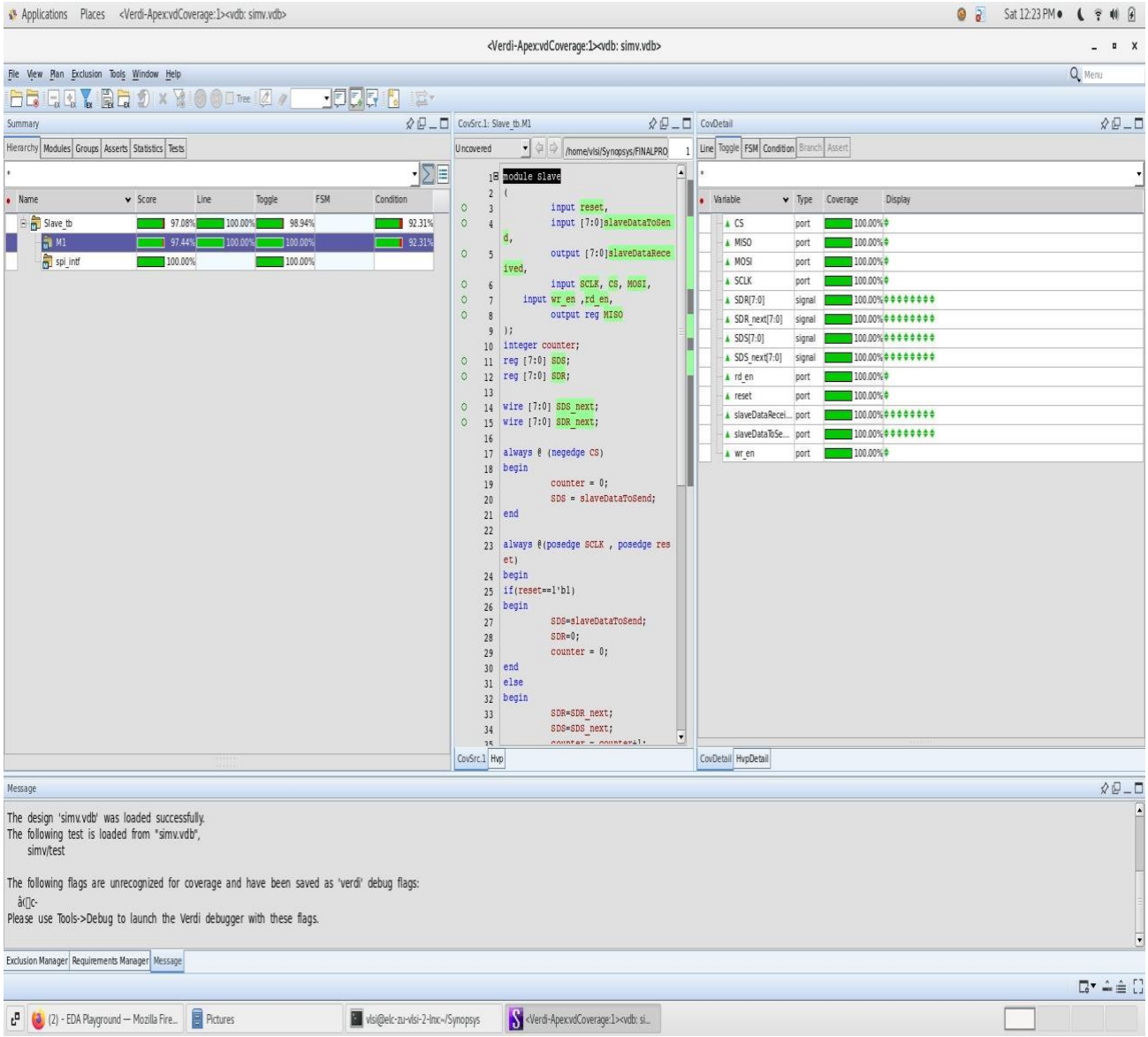


3. Creation of coverage report

   urg -dir simv.vdb

4. Viewing the created coverage report

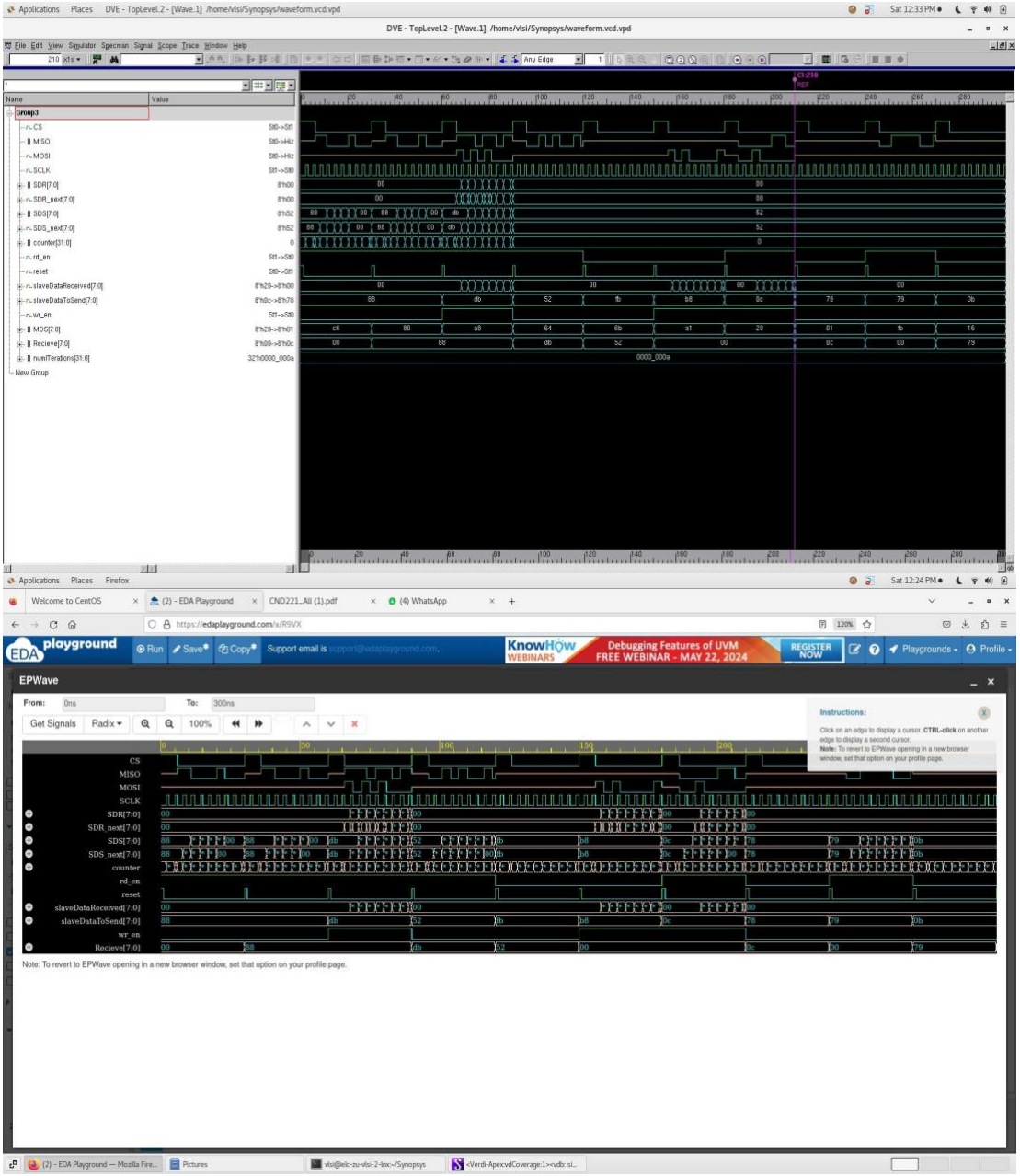
   Verdi –cov –covdir simv.vdb &



5. Viewing the created waveform.vcd from running simulation

dve &

from the menu on the left select open database file and browse to select waveform.vcd (created .VCD file in this case) expand the top testbench and select port/signal u want to view and right click the select add to wave



4. References

[1] https://github.com/Abd-ELrahmanHamza/Serial-Peripheral-Interface/tree/master